

Plan de la démo — résumé

Construire une application de gestion de **CRA** avec des assistants IA : React + FastAPI, 17 user stories, de l'outillage avant le code. Le détail — prompts et commandes — est dans [PLAN.MD](#).

L'application

Un **compte rendu d'activité** est la déclaration mensuelle d'un consultant : ce qu'il a fait, jour par jour, et sur quelle mission. C'est la pièce qui sert ensuite à facturer le client et à suivre les congés.

Deux rôles, deux parcours :

- **Le consultant** ouvre le calendrier du mois et remplit chaque jour ouvré : une journée sur une mission, ou une journée partagée en deux demi-journées entre deux clients, ou une absence — congé payé, RTT, maladie, sans solde, formation. Un récapitulatif lui donne en direct ses totaux par mission et par type d'absence, et lui signale les jours encore vides. Quand le mois est complet, il le soumet.
- **Le manager** crée les missions, y affecte ses consultants, et retrouve dans sa liste les CRA soumis par son équipe. Il ouvre chacun, le valide — le CRA est alors verrouillé — ou le refuse avec un motif obligatoire, ce qui le renvoie en brouillon chez le consultant pour correction.

Le cycle de vie tient en une ligne : **Brouillon** → **Soumis** → **Validé**, ou **Soumis** → **Refusé** → **Brouillon**.

Deux règles structurent tout le métier : un jour ne peut jamais totaliser plus d'**une** journée (d'où les demi-journées), et un CRA validé n'est plus modifiable par personne. Week-ends et jours fériés français ne sont pas saisissables.

L'authentification est volontairement **bouchonnée** : choisir un profil dans une liste tient lieu de connexion. L'effort va au métier, pas à la gestion de comptes.

Phase 0 — Équiper les assistants

Avant la première ligne de code. C'est ce qui rend le code homogène quel que soit l'outil.

Élément	Rôle
Spec Kit	porte le cycle par story ; sa <i>constitution</i> impose nos règles et nos agents
MCP Chrome DevTools	vérifier l'application qui tourne, pas seulement les tests
AGENT.md	mémoire projet unique et générique, avec un pointeur par outil
4 règles Do / Don't	pratiques et anti-patterns, backend et frontend
10 skills	modes d'emploi : endpoint, modèle, écran, design system, tests, revue, documents
5 agents	business-analyst , architect , fastapi-dev , react-dev , cra-reviewer

Tout existe en double : `.claude/` pour Claude Code, `.github/` pour GitHub Copilot.

Phase 1 — Le socle

Deux projets qui démarrent et se parlent, sans une seule fonctionnalité métier : un backend FastAPI, un frontend React, un écran qui affiche l'état de l'API. Plus un jeu de données de démonstration — deux consultants, un manager, quelques missions.

L'intérêt de cette phase est ailleurs que dans le code produit. C'est le moment où l'on tranche les quelques choix qui coûteraient cher à reprendre une fois dix écrans écrits :

- **la forme des messages d'erreur**, identique partout et en français, puisque l'interface les affiche tels quels ;
- **le nommage des données échangées** entre Python et TypeScript, que les deux langages écrivent différemment ;
- **le niveau de test exigé**, inscrit dans l'outillage plutôt que dans une consigne — la suite échoue sous 70 % de couverture ;
- **les interdits du langage**, appliqués par le linter et non par la bonne volonté.

Rattraper l'un de ces choix après cinq stories, c'est une reprise sur toute la base de code.

Phases 2 à 6 — Les 17 stories, dans l'ordre

Chaque story cible **une** fonctionnalité et ne dépend que des précédentes.

Phase	Stories
2 — Identité	connexion par profil · identité affichée et déconnexion
3 — Missions	créer · lister et filtrer · affecter des consultants · clôturer
4 — Saisie	calendrier du mois · saisir une journée · demi-journées · absence · récapitulatif
5 — Workflow	soumettre · annuler la soumission · lister les CRA en attente · valider · refuser avec motif
6 — Restitution	tableau de bord du consultant

La boucle, identique pour chaque story

```
business-analyst → architect → tests d'API → fastapi-dev || react-dev → cra-reviewer
la story est-      contrat    un test par    backend et frontend    relecture +
elle implémentable ? d'API    critère        en parallèle        navigateur
```

Deux principes gouvernent cette boucle : **les tests d'API sont écrits avant le code**, à partir des critères d'acceptation ; et **un agent par côté**, chacun dans son dossier, sur un contrat figé avant qu'ils démarrent.

Phase 7 — La démonstration

Scénario complet dans le navigateur : Jean saisit son mois et le soumet, Paul refuse avec un motif, Jean corrige et resoumet, Paul valide. Puis vérification des refus d'accès par appels directs à l'API — un bouton masqué ne prouve rien.

Ce que la démo cherche à montrer

Non pas qu'une IA écrit du code, mais que **le dispositif la rattrape** : un agent qui refuse une conception au nom d'une règle, un relecteur qui trouve un défaut que toute une suite verte ignorait, une revue qui bloque une story parce qu'on ne saurait pas écrire son test.